Claude 도구 위험 유형 분류 (심화)

Oct 17, 2026 · @Tion · v2 개정 Oct 26, 2026

Claude 도구의 위험 행동은 9개 유형으로 나뉘고, 정규식과 파일 파싱으로 안정적으로 잡히는 것은 R1·R2·R3·R5·R9이며 R4·R6·R7·R8은 맥락 판단이 필요해 사람 라벨링과 AI 보조를 함께 써야 한다.

v2 개정 사항

1. **정답 라벨 정리:** 정답은 사람 파일 라벨로 하고, 실측 결과는 "확인 수준" 3단계로 쓴다. (정확도 측정 계획)
2. **재현율 분모 한정:** 단계별 탐지 대상 유형만 분모로 잡고, 범위 밖 동작은 "범위 외"로 따로 센다. (정확도 측정 계획)
3. **R9 신설:** 에이전트 설정·권한 변경. 1차 범위에 포함한다. (8개 → 9개 유형)
4. **R2 탐지 대상 확대:** 사용자 설정뿐 아니라 플러그인 내부 `hooks/` 폴더를 본다.
5. **R8 두 갈래로 분리:** 숨은 행동 지시 / 위험 행동 유도 지시(악의 무관).
6. **라이선스 기준 보완:** NOASSERTION 확인 절차, 콘텐츠 동일조건(CC-BY-SA) 계열, 듀얼 라이선스 예.

본문에서 바뀐 부분은 왼쪽 파란 선과 v2 표시로 구분했다.

9개 유형 한눈에 v2

유형	위험 핵심	주로 보는 파일	탐지 방식	탐지 난이도	도입 단계
R1 원격 코드 실행형 설치	검증 불가한 외부 코드를 즉시 실행	README, install.sh, package.json	정규식	낮음	1차
R2 자동 실행 훅의 셸 명령	사용자 승인 없이 반복 실행	hooks 설정, **플러그인 hooks/ 폴더**, 훅 스크립트	JSON 파싱 + 정규식	낮음	1차
R3 민감 정보 접근	키·토큰·세션 탈취 경로	스크립트 전반	경로 패턴 + 목적 대조	중간	1차
R4 외부 전송	데이터 유출	스크립트, 서버 코드	도메인 + 전송 내용	높음	후속
R5 파괴적·광범위 파일 작업	도구 범위를 넘는 시스템 영향	스크립트	정규식	낮음	1차
R6 난독화·은닉	사람의 검토를 피한 흔적	스크립트 전반	인코딩 패턴	높음 (오탐 많음)	후속
R7 과도한 MCP 권한	최소 권한 원칙 위반	.mcp.json, 서버 코드	JSON 파싱 + 권한 범위 비교	중간	후속
R8 지시문 속 의심 문장	AI에게 숨은 행동이나 위험 행동을 시키는 지시	SKILL.md, commands/*.md	문구 패턴 + AI 보조	높음	3단계

R9 에이전트 설정·권한 변경	사용자 확인 절차와 에이전트 동작을 지속적으로 바꿈	설치 스크립트, 설치기 코드, 플러그인 매니페스트	경로 패턴 + JSON 파싱	낮음	1차

전제: 왜 따로 봐야 하나

- **사용자 권한으로 실행된다.** 플러그인·훅·MCP 서버는 설치한 사람의 계정 권한으로 돌아서, 사용자가 읽을 수 있는 파일(SSH 키, 클라우드 자격 증명)을 그대로 읽을 수 있다.
- **승인 없이 실행되는 경로가 있다.** 훅은 이벤트마다 자동 실행되고, 패키지의 설치 후 스크립트(postinstall)는 설치 시점에 실행된다.
- **지시문이 곧 실행 코드다.** SKILL.md와 명령 파일은 AI에게 내리는 지시라서 코드 검사만으로는 위험 행동이 보이지 않고, 기존 보안 스캐너도 이 영역을 거의 다루지 않는다.
- **결과는 판결이 아니라 신호다.** "안전"이라고 단정하지 않고 "확인한 위험 요소"로 표시한다. 발표 7장과 예상 질문의 원칙과 같다.

설치·실행 경로 (R1, R2)

R1 원격 코드 실행형 설치

- **위험 메커니즘:** 설치 중 내려받은 스크립트를 내용 확인 없이 바로 셸에 넘긴다. 받는 시점의 내용을 검증할 수 없어서, 서버나 도메인이 탈취되면 같은 설치 명령이 악성 코드를 실행한다.
- **탐지 신호:** `curl ... | sh`, `wget ... | bash`, `irm ... | iex`, `bash <(curl ...)`. README, 설치 스크립트, package.json의 스크립트에서 찾는다.
- **흔한 오탐:** 널리 쓰이는 도구의 공식 한 줄 설치도 같은 형태다. 도메인이 공식 출처인지로 구분한다.
- **심각도 보정:** 도메인이 저장소 소유자나 공식 배포처면 한 단계 낮추고, 단축 URL이나 IP 직접 지정이면 높인다. 고정 버전과 체크섬 검증이 있으면 낮춘다.

R2 자동 실행 훅의 셸 명령 v2

- **위험 메커니즘:** 훅은 지정한 이벤트(프롬프트 제출, 도구 호출 전후 등)마다 사용자 확인 없이 명령을 실행한다. 한 번 심어 두면 반복 실행된다.

> - **탐지 대상 위치:** 사용자 `settings.json`의 hooks 항목만 보면 안 된다. 플러그인은 훅을 사용자 설정이 아니라 **플러그인 안의 `hooks/` 폴더**에 담아 오므로, 이 폴더의 설정 파일과 훅 스크립트를 함께 읽는다. 설치 형태가 플러그인인 도구가 많아서 이 위치를 빼면 훅을 통째로 놓친다.

- **탐지 신호:** hooks 설정의 command 값에 `bash -c`, `eval`, `curl`, 외부 스크립트 호출이 있는 경우. 어떤 이벤트에 걸리는지도 함께 기록한다. 매 프롬프트나 매 도구 호출에 걸릴수록 노출 빈도가 높다.
- **흔한 오탐:** 파일 저장 뒤 포매터·린터를 돌리는 정상 자동화.
- **심각도 보정:** 고정된 로컬 명령이면 낮추고, 프롬프트나 파일 내용 같은 외부 입력을 셸에 그대로 넘기면 높인다.

민감 정보와 외부 전송 (R3, R4)

R3 민감 정보 접근

- **위험 메커니즘:** 자격 증명과 세션 정보를 읽으면 계정 탈취로 이어진다. 읽기만 해도 이후 전송 단계와 결합하면 유출이 된다.
- **탐지 신호:** `.env`, `~/.ssh`, `~/.aws`, `~/.gnupg`, 브라우저 저장소(쿠키·로그인 DB), 토큰·API 키 환경변수 읽기.
- **핵심 판단:** 도구의 설명과 무관한 경로를 읽을수록 가중치가 높다. 코드 정리 도구가 SSH 키 경로를 읽으면 이상하다.
- **흔한 오탐:** 클라우드 배포 도구의 `~/.aws` 읽기, Git 도구의 `~/.ssh` 읽기는 정상 동작이다.
- **심각도 보정:** 도구 분류별 허용표(배포→`~/.aws`, Git→`~/.ssh`)와 일치하면 낮추고 불일치하면 높인다.

R4 외부 전송

- **위험 메커니즘:** 파일이나 환경변수 내용을 요청 본문에 실어 외부 서버로 보낸다.
- **탐지 신호:** 알 수 없는 도메인으로의 HTTP 요청, `curl -d`, `requests.post`, `fetch`의 POST 본문.
- **기록 원칙:** 텔레메트리와 유출은 코드만으로 구분하기 어렵다. 그래서 "목적지 도메인"과 "보내는 내용"을 함께 기록한다.
- **흔한 오탐:** 사용 통계·오류 보고, 공식 API 호출.
- **심각도 보정:** 알려진 공식 도메인이면 낮추고, 본문에 파일·환경변수 내용이 들어가면 높인다. 사용자에게 고지하고 끌 수 있는지도 확인한다.

시스템 영향과 은닉 (R5, R6)

R5 파괴적·광범위 파일 작업

- **위험 메커니즘:** 도구 설명의 범위를 벗어나 시스템 전체에 영향을 준다.
- **탐지 신호:** `rm -rf /`, `rm -rf ~`, `rm -rf $HOME`, 홈 디렉터리 전체 탐색·덮어쓰기, 셸 프로필(`.bashrc` 등) 수정, `chmod 777`, `sudo`.
- **흔한 오탐:** `rm -rf ./build`처럼 작업 디렉터리 안의 정리, PATH 추가를 위한 설치 스크립트의 프로필 수정.
- **심각도 보정:** 대상 경로가 도구 작업 디렉터리 안이면 제외하고, 프로필 수정은 사용자 고지 여부에 따라 나눠 본다.

> **범위 구분** v2 · `~/.claude` 등 에이전트 설정 위치에 쓰는 작업은 파괴적이지 않아 R5 신호에 걸리지 않는다. 이 작업은 R9로 분리해 본다. 반대로 지시문이 AI에게 파일 삭제를 시키는 경우는 R5 정규식에 걸리지 않으므로 R8-b에서 다룬다.

R6 난독화·은닉

- **위험 메커니즘:** 사람이 읽고 검토하는 것을 피하려는 흔적이다.
- **탐지 신호:** base64 디코드 후 실행, 의미 없는 변수명으로 압축된 코드, 긴 인코딩 문자열.
- **흔한 오탐 (많음):** 번들된 JS, 최소화(minified) 파일, 이미지·폰트의 base64. 정상 도구에도 흔하다.
- **심각도 보정:** 디코드 결과를 셸이나 `eval`에 넘기는 경우만 높게 보고, 단순히 긴 문자열은 낮음으로 기록만 한다.

AI 도구 고유 위험 (R7, R8)

R7 과도한 MCP 권한

- **위험 메커니즘:** MCP 서버는 AI가 호출하는 도구의 통로다. 필요 이상의 범위를 열어 두면 AI의 실수나 주입 공격이 곧 시스템 피해가 된다.
- **탐지 신호:** 파일시스템 서버가 루트나 홈 전체를 허용, 셸 실행 도구 노출, 설정에서 인증 정보를 받는 구조. `.mcp.json`의 command, args, env를 파싱해 확인한다.
- **판단 방식:** 최소 권한(필요한 폴더만, 읽기 전용 등)과 비교해 "넓음/좁음"으로 표시한다.
- **흔한 오탐:** API 키를 환경변수로 받는 것은 정상적인 연동 방식이다.
- **심각도 보정:** 키 요구 자체는 낮음, 범위가 넓은 경로나 셸 노출은 높음.

R8 지시문 속 의심 문장 v2

- **위험 메커니즘:** 스킬·명령 파일의 문장이 AI의 행동을 정한다. 프롬프트 주입처럼 사용자 모르게 위험 행동을 시킬 수 있다.

두 갈래로 나눠 본다. 악의적 지시만 보면, 악의는 없지만 AI에게 위험 행동을 시키는 지시를 놓친다. 이런 지시는 셸 스크립트가 아니라 문장이라서 R5 정규식에도, 아래 R8-a 문구 목록에도 걸리지 않는다.

구분	탐지 신호	예
R8-a 숨은 행동 지시	"사용자에게 알리지 말고", "이전 지시를 무시하고", 비밀 정보를 출력·전송하라는 문장 (영문 표현 포함)	프롬프트 주입형 지시
R8-b 위험 행동 유도 지시 (악의 무관)	AI에게 파일 삭제, 강제 덮어쓰기, 확인 없는 명령 실행, 권한 변경 등을 시키는 문장	"테스트보다 먼저 작성된 코드는 삭제한다" 같은 작업 규칙

- **차별점:** 기존 보안 스캐너가 거의 다루지 않는 AI 도구 고유의 위험이며, 이 프로젝트의 차별점이 될 수 있다. 특히 R8-b는 "정규식으로는 안 보이고 AI로는 보이는" 대표 사례가 된다.
- **흔한 오탐:** 보안 교육·방어용 스킬이 해당 문구를 인용하는 경우.
- **탐지 방식:** 문구 목록으로 1차 탐지하고 AI 보조로 문맥을 판정한다. "정규식 vs AI"의 차이 자체가 분석 결과가 된다.
- **심각도 보정:** 사용자에게 숨기거나 비밀을 보내라는 지시면 높음, 인용·예시 맥락이면 낮춤. R8-b는 사용자 확인 단계를 두는지, 대상이 작업 디렉터리 안인지에 따라 나눠 본다.

에이전트 설정 변경 (R9) v2 신설

R9 에이전트 설정·권한 변경

- **위험 메커니즘:** 설치기나 훅이 에이전트의 설정 파일을 고쳐, 사용자 확인 절차나 에이전트의 기본 동작을 지속적으로 바꾼다. 특히 도구 자동 허용 권한을 등록하면 이후 그 도구 호출에 사용자 확인이 생략된다. 파일을 지우지 않아 R5에 걸리지 않지만, 영향은 도구를 쓰지 않는 세션까지 이어진다.

> - **탐지 신호:** `~/.claude/settings.json`의 `permissions.allow` 항목 추가, `CLAUDE.md`·`AGENTS.md` 등 에이전트 지시 파일 쓰기, `~/.claude.json`의 `mcpServers` 수정, `~/.claude` 아래 바로가기·기록 파일 생성. 설치 스크립트와 설치기 코드에서 이 경로와 설정 키를 찾는다.
> - **흔한 오탐:** 사용자에게 변경 내용을 보여 주고 확인받은 뒤 MCP 서버를 등록하는 정상 설치 절차.
> - **심각도 보정:** 권한 자동 허용 추가는 높음. 사용자 고지와 동의가 있으면 한 단계 낮추고, 구분 표시(마커)로 감싼 섹션을 쓰고 제거 명령으로 되돌릴 수 있으면 한 단계 더 낮춘다. 전역 설정(홈)이 프로젝트 설정보다 높다.

표준 분류 매핑

각 유형을 널리 쓰이는 분류 체계에 대응시키면 발표와 보고서에서 근거로 쓸 수 있다. 번호는 기억에 기반한 것이므로 인용 전에 원문(MITRE ATT&CK, CWE, OWASP LLM Top 10)과 대조해야 한다.

유형	MITRE ATT&CK	CWE	OWASP LLM Top 10
R1 원격 코드 실행형 설치	T1105 Ingress Tool Transfer	CWE-494 무결성 검증 없는 코드 다운로드	—
R2 자동 실행 훅	T1059 Command and Scripting Interpreter	CWE-78 OS 명령 주입	LLM06 Excessive Agency
R3 민감 정보 접근	T1552 Unsecured Credentials	CWE-522 불충분한 자격 증명 보호	—
R4 외부 전송	T1041 Exfiltration Over C2 Channel	CWE-200 민감 정보 노출	LLM02 Sensitive Information Disclosure
R5 파괴적 파일 작업	T1485 Data Destruction, T1546.004 Unix Shell Configuration Modification	CWE-732 중요 자원에 대한 잘못된 권한	—
R6 난독화	T1027 Obfuscated Files or Information	CWE-506 내장된 악성 코드	—
R7 과도한 MCP 권한	—	CWE-250 불필요한 권한으로 실행	LLM06 Excessive Agency
R8 지시문 속 의심 문장	—	—	LLM01 Prompt Injection
R9 에이전트 설정·권한 변경	원문 대조 후 기입	원문 대조 후 기입	LLM06 Excessive Agency (후보)

심각도 보정과 종합 판정

유형별 기본 심각도에 세 가지 보정을 거친 뒤, 도구 하나의 신호 목록을 낮음·보통·주의 중 하나로 줄여 슬라이드에 표시한다.

1. **목적 대조:** 도구의 설명과 맞는 행동이면 한 단계 낮추고, 맞지 않으면 한 단계 높인다. 예를 들어 배포 도구가 `~/.aws`를 읽으면 낮추고 마크다운 정리 도구가 읽으면 높인다.
2. **출처 신뢰:** 접속 도메인이 저장소 소유자나 공식 배포처면 한 단계 낮춘다.

3. **조합 규칙:** 같은 파일이나 함수 안에서 R3(읽기)와 R4(전송)가 함께 나오면 high로 올린다.

표시	조건	표시 문구
주의	보정 후 high 신호가 1개 이상	확인한 위험 요소가 있어 설치 전 직접 검토 필요
보통	high 없이 medium 신호가 1개 이상	일부 위험 요소 확인
낮음	medium 이상 신호 없음	확인한 범위에서 위험 요소 없음

"낮음"은 안전하다는 뜻이 아니다. 스캐너가 본 규칙 범위 안에서 신호가 없었다는 뜻이며, 화면에도 "확인한 위험 요소"라는 표현을 쓴다. 임계값은 라벨링 결과를 보고 조정한다.

정확도 측정 계획 v2

정답 기준은 사람이 같은 파일을 보고 "이 신호는 실제 문제인가"를 판정한 라벨이다. 정밀도는 스캐너가 경고한 것 중 실제 문제의 비율이고, 재현율은 사람이 문제라고 본 것 중 스캐너가 잡은 비율이다. 유형별로 따로 계산한다.

실측 결과의 역할: 정답이 아니라 확인 수준

도구를 설치해 실제로 실행해 본 동작(실측)은 정답 자체로 쓰지 않는다. 짧은 실측에서는 발동하지 않은 동작이 보이지 않기 때문이다. 훅은 특정 이벤트에서만 실행되고, 텔레메트리를 일정 기간 모았다가 보내는 도구는 몇 분짜리 실행으로는 전송이 잡히지 않는다. 실측에서 안 보였다는 이유로 헛경보로 세면 정밀도가 실제보다 낮게 나온다.

확인 수준	뜻	집계 처리
실측 확인	실행 중 해당 동작을 직접 관찰함	사람 라벨과 함께 "실측 확인"으로 표시
실측 미발동 (조건 미충족 가능)	관찰하지 못했으나 실측 중 발동 조건이 충족되지 않았을 수 있음	사람 파일 라벨을 따른다. 헛경보로 세지 않는다
실측 반증	발동 조건을 충족시켰는데도 해당 동작이 없음	사람 라벨을 다시 검토한 뒤 헛경보로 처리

재현율의 분모

재현율은 그 단계에서 탐지 대상인 유형으로 분류된 정답만 분모로 잡는다. 1차 범위 밖 유형(R4 등)에 해당하는 실측 동작은 "범위 외"로 따로 센다. 그렇지 않으면 스캐너 성능 문제가 아니라 범위 문제 때문에 재현율이 낮게 나온다. "범위 외" 건수는 후속 유형의 도입 시기를 앞당길지 판단하는 근거로 쓴다. 실제로 사전 실측(measure36)에서 관찰된 부작용 상당수는 텔레메트리, 곧 R4에 해당했다.

아래 수치는 측정 전의 **가설**이며, 샘플 5개로 확인하면 유형 간 차이의 방향 정도만 볼 수 있다. 결과에는 비율과 함께 건수를 같이 적는다.

유형	예상 정밀도	예상 재현율	이유
R1	높음	높음	형태가 고정되어 정규식으로 정확히 잡힘
R2	높음	중간	훅 설정은 구조화되어 있으나 스크립트 안의 호출은 놓칠 수 있음. 플러그인 `hooks/` 폴더를 포함해야 이 수준이 나옴
R3	중간	중간	경로는 잡히나 목적 일치 판단은 사람 몫

R5	중간	높음	좁은 범위 삭제와 구분이 필요
R9	높음	중간	설정 경로와 키가 정해져 있어 파싱이 확실하나, 번들된 설치기 코드 안의 쓰기는 놓칠 수 있음
R7	높음	높음	JSON 구조라 파싱이 확실함
R4	낮음~중간	낮음	텔레메트리와 유출 구분이 어려움
R6	낮음	중간	정상 도구에도 인코딩 문자열이 많음
R8	낮음	낮음	문장 의도 판정이 모호하여 AI 보조가 필요. R8-a와 R8-b를 따로 계산

- **1차 범위는 R1, R2, R3, R5, R9로 한다.** v2 정규식과 파일 파싱만으로 가능하고 라벨링 기준이 명확하다.
- R8은 3단계에서 AI 보조로 시도해 "정규식 vs AI"의 차이를 비교한다.
- 스캐너는 저장소 파일을 읽기만 한다. 탐지를 위해 코드를 실행하면 스캐너 자체가 위험해진다.

의존성·라이선스 축 (2차 확장)

행동 위험(R1~R9)과 별개로 의존성 취약점과 라이선스를 각각 낮음·보통·주의로 판정하고, 전체 표시는 세 축 중 가장 높은 값으로 한다. 점수를 합산하지 않는 이유는 근거가 보여야 "위험도를 믿을 수 있나"에 답할 수 있기 때문이다.

세 축 구조

축	묻는 것	입력 데이터	출처	도입 순서
행동	이 도구가 내 컴퓨터에서 뭐를 하나	저장소 파일	규칙 R1~R9	1차
라이선스	우리 저장소 라이선스와 충돌하거나 의무가 생기나	의존성의 라이선스 표기 (SPDX ID)	SBOM, 패키지 메타데이터	2차-첫 번째
의존성	쓰는 라이브러리에 악용 가능한 취약점이 있나	package.json, requirements.txt 또는 SBOM	OSV.dev(취약점·수정 버전), EPSS(FIRST), CISA KEV	2차-두 번째

의존성 위험: CVSS + EPSS + KEV

- **CVSS:** 취약점이 얼마나 심각한지 나타내는 0~10 점수.
- **EPSS:** 향후 30일 안에 실제로 악용될 확률(0~1).
- **KEV:** CISA가 실제 악용을 확인해 등재한 취약점 목록.
- **결합 이유:** CVSS만 보면 높은 점수가 너무 많아 우선순위가 안 선다. 심각하지만 아무도 안 쓰는 취약점과 지금 악용되는 취약점을 EPSS와 KEV로 구분한다.

단계	조건 (초안)
주의	KEV 등재, 또는 CVSS 7.0 이상이면서 EPSS 0.1 이상
보통	CVSS 7.0 이상, 또는 EPSS 0.01 이상

낮음	그 외

- **맥락 보정:** 개발 전용 의존성(devDependencies)은 한 단계 낮춘다. 취약한 기능을 도구가 실제로 호출하는지(도달 가능성)는 정적 분석으로 확정하기 어려워 "AI 추정"으로 표시한다.
- **임계값**은 측정 전의 가설이며 라벨링 결과로 조정한다.

라이선스 위험 v2

우리 저장소는 MIT 라이선스(예정)이고 웹 서비스로 제공한다. 이 두 조건이 기준이 된다.

라이선스 계열	예	우리 저장소와의 관계	기본 단계
허용형	MIT, BSD, Apache-2.0	결합 가능, 저작권 고지 의무	낮음
약한 카피레프트	LGPL, MPL-2.0	라이브러리로 쓰면 대체로 가능, 수정분 공개 의무	보통
강한 카피레프트	GPL-2.0, GPL-3.0	결합·배포 시 전체 소스 공개 의무가 퍼질 수 있어 MIT와 충돌 가능	주의
네트워크 카피레프트	AGPL-3.0	웹 서비스로 제공만 해도 소스 공개 의무가 생길 수 있어 우리 구조에 직접 해당	주의
콘텐츠 동일조건 (share-alike)	CC-BY-SA-4.0	소프트웨어용이 아닌 저작물 라이선스. 스킬 문서 등을 고쳐 쓰면 같은 라이선스로 공개해야 할 수 있음	보통, 사람 확인
라이선스 없음·불명	표기 없음, NOASSERTION (LICENSE 파일 확인 후)	사용 권리가 불분명	주의
용도 제한형 (AI 모델 라이선스 등)	Llama 커뮤니티 라이선스, OpenRAIL 계열	사용 용도·규모 제한	보통 이상, 사람 확인

- AI 모델 라이선스는 "오픈소스와 AI를 결합할 때 생기는 충돌"이라는 프로젝트 주제와 직접 연결된다.
- 실제 의무는 연결 방식(동적·정적 링크)과 배포 여부에 따라 달라지므로 자동 판정은 "검토 필요" 수준으로 표시한다. 법률 자문이 아니다.

> - **NOASSERTION 처리:** 바로 "주의"로 판정하지 않고 저장소의 LICENSE 파일을 직접 확인한다. GitHub가 이 값을 내는 경우는 라이선스가 없을 때뿐 아니라, 라이선스 파일이 여러 개이거나 표준 문구가 아니라서 자동 인식에 실패했을 때도 많다. 확인 결과 표준 라이선스면 그 계열로 판정하고, 정말 없으면 "주의"로 둔다.
> - **듀얼 라이선스**는 사용자가 고를 수 있는 선택지를 함께 표시한다. 저장소에 라이선스 파일이 둘 이상 있으면(예: Apache-2.0과 MIT 파일이 함께 있는 경우) 저장소 메타데이터에 하나만 나와도 둘 다 기록한다.

SBOM 입력과 표준

- CycloneDX와 SPDX는 구성요소의 이름·버전·라이선스를 담는 SBOM(소프트웨어 구성 목록) 표준 규격이다. JSON 입력을 지원하면 국제 표준에 맞는 입력 방식이 된다.
- **우리가 쓰는 필드:** 구성요소 이름, 버전, purl(패키지 식별자), 라이선스. purl로 취약점 데이터베이스를 조회한다.
- 대부분의 Claude 도구 저장소에는 SBOM 파일이 없으므로 package.json이나 requirements.txt에서 직접 읽거나 생성하는 단계가 필요하다.

조치 방안 (Remediation)

데이터에서 나온 것과 AI가 쓴 것을 화면에서 구분해 보여 준다. 판정의 확인 수준 표시(AI 추정 / 사람 확인)와 같은 원칙이다.

종류	내용	근거 표시
수정 버전	"X 버전 이상으로 업그레이드"	데이터 기반 (OSV 등의 수정 버전 정보)
임시 조치	패치가 없을 때 우회·완화 방법	AI 추정
라이선스 조치	대체 라이브러리 후보, 의존성 제거 검토	AI 추정

도입 순서와 한계

1. **라이선스 축:** 구현이 가장 쉽고 우리 MIT 라이선스와 바로 연결된다.
2. **의존성 취약점(OSV):** 패키지명과 버전으로 조회한다.
3. **EPSS·KEV 결합:** 우선순위를 정한다.
4. **조치 방안:** 수정 버전 안내부터 추가한다.

- 취약점이 있다는 것과 그 취약점이 실제로 호출된다는 것은 다르다. 이 축의 결과는 "존재"를 보여 주는 것이지 "악용 가능"을 확정하는 것이 아니다.
- 외부 데이터베이스 값이라 스캐너 정밀도보다 데이터의 최신성이 정확도를 좌우한다.
- 스캐너는 여전히 저장소 코드를 실행하지 않고, 외부로 보내는 것은 패키지명과 버전뿐이다.
- 행동 축과 달리 외부 조회가 필요하다. 네트워크가 안 되면 이 축은 "조회 실패"로 표시하고 낮음으로 처리하지 않는다.